PALET: auditoría técnica y metodológica y plan de mejora

Del "buscador + clasificador" actual a una herramienta de revisión de alcance asistida por IA defendible ante cualquier *referee*

Proyecto: PALET (Prompt-based Automated Literature Extraction Tool), ISABEL — GA 101177687

Base revisada: carpeta `palet/` (commit `da90731`), corpus del capítulo y los dos informes de *referees* (`ISABELCHAPTER REV.docx`)

Fecha: 11 de noviembre de 2026

Destinatarios: Ricardo Martínez de Vega Perancho, Fernando Rubiera Morollón y el agente implementador (Claude Fable 5.2)

Nota para el agente implementador

Este documento es a la vez un diagnóstico y una especificación. Lee primero el Resumen ejecutivo (§1) y las Instrucciones operativas (§11). Después ejecuta los paquetes de trabajo de §8 *en el orden indicado*, uno a uno, cumpliendo sus criterios de aceptación antes de pasar al siguiente. Cada afirmación sobre el código remite a un fichero; verifícala antes de cambiar nada.

Índice

1 Resumen ejecutivo **2**

2 Alcance y fuentes revisadas **4**

3 Diagnóstico empírico del corpus del capítulo **4**
3.1 Cribado heurístico por RQ 4
3.2 Hallazgos 4

4 Comentarios de los referees → causa raíz → solución **5**

5 Fortalezas (conservar) **6**

6 Debilidades por módulo **7**
6.1 ARQ (`scripts/arq_module.py`, `prompts/arq_system_v3.txt`) 7
6.2 QSP (`scripts/qsp_module.py`, `prompts/qsp_system_v2.txt`) 7
6.3 SE (`scripts/se_module.py`) 8
6.4 MAP (`scripts/map_module_v2.py`, `prompts/map_system_v2.txt`) 8
6.5 PRISMA (`prisma_validator.py`) 8
6.6 Síntesis (`scripts/synthesis_module.py`, `prompts/synthesis_system.txt`) . . . 9
6.7 Informe y BibTeX 9
6.8 GUI, orquestación e infraestructura 9

7 Arquitectura objetivo: PALET 2.0 **10**
7.1 Principios de diseño 10
7.2 Flujo 10
7.3 Estructura de directorios propuesta 10

8 Paquetes de trabajo (especificación para el agente) **11**

9 Plan para la revisión del capítulo **17**
9.1 Dos opciones para el corpus 17
9.2 Pasos (opción A o intermedia) 17
9.3 Plantillas de texto para el capítulo (inglés; completar tras la re-ejecución) 18

10 Qué significa "una herramienta de 10" **18**

11 Instrucciones operativas para el agente implementador **19**

A Lista de errores con ubicación **19**

B Esquemas de datos (Pydantic v2) **20**

C Esqueleto del *prompt* de cribado **22**

D Códigos de razón de exclusión (orden de precedencia) 22

E Propuesta de protocolo para el capítulo ISABEL (a validar por los autores) 23

F Referencias y fuentes externas 23

1 Resumen ejecutivo

Veredicto

La idea de PALET es buena y la base de ingeniería es razonable para un primer prototipo: fases con nombre, artefactos JSON por fase, *prompts* cuidados y una GUI usable. Pero los dos *referees* han dado en el clavo, y el problema no es cosmético sino **estructural**. Hoy PALET es, en esencia, **un buscador seguido de un clasificador temático**; **no es todavía un *pipeline* de revisión de alcance**. En concreto:

1. **No criba con criterios de elegibilidad.** El LLM decide si un *abstract* "responde a alguna RQ", pero los criterios de inclusión/exclusión que genera ARQ nunca se aplican. Por eso entran estudios sobre China, Perú o pérdidas de fósforo.
2. **No deja rastro auditable.** No hay razones de exclusión, la cadena de búsqueda real no se archivó, la fecha de búsqueda no se registra y los errores del modelo se cuentan como exclusiones.
3. **No mide su propia fiabilidad.** No hay comparación con cribado humano, ni recall de la búsqueda, ni estabilidad entre ejecuciones. Afirmar que el procedimiento es "objetivo" o que hace un "barrido completo" no se sostiene, y los *referees* lo han visto.
4. **La síntesis no está anclada a las fuentes.** No exige citas por afirmación ni comprueba que la fuente diga lo que se le atribuye: es exactamente el fallo M4/M5 del Referee 2.
5. **La búsqueda no está diseñada para RQs heterogéneas.** Se usa una sola cadena booleana para todas las RQs (en OpenAlex, que además busca en el texto completo), se trunca en silencio a los primeros N resultados por relevancia y no se guarda el número total de resultados encontrados.

Matiz importante sobre el capítulo

Los 142 artículos del capítulo **no salieron del código Python actual**. Salieron del flujo de primavera de 2025: una exportación CSV de Scopus con 235 registros (`Prueba Final/Final_Fernando_3.1.csv`), clasificada en Excel como *Admitido/No admitido* con una RQ por artículo (`Prueba_Final_clasificado.xlsx`). El código se escribió después para replicar ese proceso (el meta-prompt fija como objetivo "~142 papers"). Aun así, **el código actual reproduciría los mismos fallos**, y añade otros (§6). Las dos cosas hay que arreglarlas a la vez: la herramienta y la forma de contarla en el capítulo.

Evidencia cuantitativa (corpus del capítulo)

- 235 registros de Scopus → 234 en la hoja de clasificación: **un registro desapareció sin rastro** ("Why levelling-up is unlikely to solve regional disparities", 2020, sin autores en la exportación).
- 142 incluidos / 92 excluidos, **sin ninguna razón de exclusión registrada**.
- Un cribado heurístico por palabras clave (título, resumen y palabras clave del autor) da lo siguiente. De los 142 incluidos, **40 (28%) no mencionan Europa, la UE ni ningún Estado miembro**, y **22** mencionan explícitamente solo contextos no europeos. Además, **65 (46%) no contienen ningún término de mercado laboral** (*employ*, job*, labo(u)r, work*, skill**) en título ni resumen.
- Hay **un duplicado no detectado**: Coaquira et al. (2022, sin DOI) y Velásquez et al. (2023, `10.13043/DYS.94.4`) son el mismo trabajo sobre Perú, con título bilingüe y fechas distintas. La deduplicación por DOI exacto o título exacto no puede detectarlo.

- La distribución está muy desequilibrada: RQ3 = 61, RQ6 = 39, RQ1 = 18, RQ2 = 13, RQ7 = 5, RQ4 = 4, RQ5 = 2. Cada artículo tiene una sola RQ, pese a que el texto del capítulo sugiere una asignación múltiple.

Puntuación actual y objetivo

Dimensión	Hoy	Objetivo	WPs clave
Ingeniería de software (estructura, robustez, tests)	5/10	9/10	0, 1, 11
Diseño y ejecución de la búsqueda	3/10	9/10	3, 4
Cribado / elegibilidad	2/10	9/10	5, 7
Validación y fiabilidad medida	0/10	9/10	6
Trazabilidad y reproducibilidad	3/10	10/10	1, 9
Síntesis anclada a evidencia	3/10	9/10	8
Cumplimiento PRISMA-ScR / RAISE	2/10	9/10	9
Usabilidad (GUI, humano en el bucle)	6/10	9/10	2, 5, 11
Global (como método publicable)	**≈3/10**	**9–10/10**	

Los 10 cambios que más mueven la aguja

1. **Protocolo primero y congelado** (**WP2**): las RQs son del investigador, no del LLM. Criterios PCC (población, concepto, contexto) explícitos y aprobados por humanos.
2. **Nueva fase SCREEN** (**WP5**): decisión por criterio, con cita textual del resumen, código de razón de exclusión y categoría *unclear* que va a revisión humana.
3. **Dos cribadores independientes más adjudicación humana** de discrepancias e *unclear* (**WP5**).
4. **Fase VALIDATE** (**WP6**): muestra estratificada cribada por dos humanos a ciegas, con sensibilidad, especificidad, precisión y κ con IC del 95%, más recall de la búsqueda frente a un conjunto de referencias conocidas.
5. **Búsqueda por bloques conceptuales y por RQ** (**WP3**), con OpenAlex restringido a título y resumen (`search.title_and_abstract`) y **API key**, adaptador de Scopus y registro del recuento total y de la fecha de búsqueda.
6. **Deduplicación difusa** por DOI, título normalizado, primer autor y año (**WP4**).
7. **Errores ≠ exclusiones** (**WP0**, **WP5**): todo registro tiene un estado y una razón.
8. **Extracción estructurada más síntesis con citas verificadas automáticamente** (**WP8**): cada afirmación lleva IDs de artículos y una cita literal que se comprueba contra la fuente.
9. **Informe PRISMA completo** (**WP9**): diagrama PRISMA 2020 con razones, *checklist* PRISMA-ScR de 22 ítems, apéndice de métodos reproducible y declaración de uso de IA conforme a RAISE.
10. **Manifiesto de ejecución y caché** (**WP1**): hash del código, IDs exactos de modelo, *hashes* de los *prompts*, parámetros, respuestas crudas de las API y de los LLM. Así una re-ejecución reproduce exactamente las decisiones.

2 Alcance y fuentes revisadas

- **Código**: `palet.py`, `app.py`, `prisma_validator.py`, `scripts/arq_module.py`, `scripts/qsp_module.py`, `scripts/se_module.py`, `scripts/map_module_v2.py`, `scripts/synthesis_module.py`, `scripts/report_generator_v2.py` (lectura parcial dirigida) y `scripts/export_bibtex.py`. Además, los *prompts* `arq_system_v3.txt`, `qsp_system_v2.txt`, `map_system_v2.txt` y `synthesis_system.txt`, `requirements.txt`, `.gitignore` y `README.md`.
- **Ejecuciones**: `runs_history.json` y salidas en `outputs/json/` (ARQ, QSP, MAP; febrero–junio de 2026).
- **Corpus del capítulo**: `Prueba Final/` (CSV Scopus, Excel de clasificación y tabla `pi_reference_table.tex`).
- **Documentos**: informes de los dos *referees* y borrador de respuesta (`ISABELCHAPTER REV.docx`), D2.3 de ISABEL y meta-prompt del agente.
- **Fuentes externas**: documentación vigente de OpenAlex y de la Scopus API, declaración conjunta Cochrane/Campbell/JBI/CEE sobre IA (RAISE) y literatura sobre cribado con LLM (§F).

No revisado: `palet_codex/` (copia anterior; parece un *fork* de marzo de 2026), `static/` y `templates/` en detalle, y las líneas 600–1334 de `report_generator_v2.py` salvo los puntos citados. La copia conectada como `Revision Bibliografia con Fernando/palet` estaba vacía; **la copia de trabajo real es** `Artículo Revision Bibliografia con Fernando/palet/`.

3 Diagnóstico empírico del corpus del capítulo

3.1 Cribado heurístico por RQ

El cálculo se hizo sobre `Prueba_Final_clasificado.xlsx`. "Menciona UE" significa que título, resumen o palabras clave contienen Europa/UE, un Estado miembro, el Reino Unido, *cohesion policy*, *smart specialisation* o NUTS. "Término laboral" significa *employ*, job*, labo(u)r, work*, skill** o *unemploy** en título o resumen. Es un indicador grueso, no un cribado: sirve para dimensionar el problema, no para decidir inclusiones.

RQ	n	% menciona UE	% término laboral	Lectura
RQ1 (transición verde)	18	78	100	razonable
RQ2 (digital/IA)	13	85	77	razonable
RQ3 (cohesión / S3)	61	74	38	cajón de sastre
RQ4 (educación)	4	100	100	muy escaso y mal asignado
RQ5 (inmigración)	2	50	100	muy escaso; 1 sobre China
RQ6 (transición justa)	39	56	49	mucho no-UE
RQ7 (lagunas de medición)	5	100	20	poco laboral
Total	142	72	54	

3.2 Hallazgos

1. **RQ3 actuó como sumidero.** Tal como está redactada ("What cohesion or smart-specialisation policies does the evidence suggest?"), no lleva ancla geográfica ni laboral. Cualquier trabajo

de desarrollo regional, en cualquier país, "responde" a ella. Eso explica que incluya a Ye et al. (China), Pycroft et al. (nivel del mar), Jedelhauser y Binder (fósforo), Han et al. (EE. UU.) o Pobedin (Rusia). **Lección para la herramienta**: el cribado debe evaluar los criterios de elegibilidad *antes* y *con independencia de* la asignación a RQs.

2. **RQ4 y RQ5 apenas tienen corpus.** La causa más probable está en la búsqueda, no en la literatura. Una cadena única centrada en transición, cohesión y especialización inteligente apenas recupera trabajos de educación o de inmigración. Es una hipótesis, porque **la cadena de Scopus no se archivó**, y eso ya es un hallazgo en sí mismo. La re-ejecución de mayo de 2026 confirma el patrón de diseño: un único bloque temático mezcla términos de varias RQs (*skills training OR ... OR artificial intelligence*) unidos con AND a otros tres bloques.
3. **No hay razones de exclusión.** PRISMA-ScR exige informar del número de excluidos y de las razones. Hoy no se pueden reconstruir.
4. **Un registro se perdió en silencio** entre el CSV y la hoja de clasificación (235 → 234).
5. **Duplicado no detectado** (Coaquira 2022 / Velásquez 2023): mismo trabajo, coautor común (Tudela), título bilingüe y revista y año distintos.
6. **Asignación de una sola RQ.** El borrador habla de artículos que "responden a" RQs, pero cada artículo quedó en una sola. Hay que decidir si la asignación es única o múltiple, declararlo y aplicarlo así en el código.

4 Comentarios de los referees → causa raíz → solución

Comentario	Causa raíz en PALET	Cambio en la herramienta	Acción en el capítulo
R1-Mayor-1 / R2-M1: modelo, bases, cadenas, fechas, cifras PRISMA	No hay manifiesto de ejecución; la cadena de Scopus no se archivó; no se registra la fecha de búsqueda; no hay razones de exclusión	**WP1** manifiesto; **WP3** registro de búsqueda (PRISMA-S); **WP9** apéndice de métodos automático	Apéndice con cadenas, fechas, modelo y cifras (plantilla en §9)
R1-Mayor-2 / R2-M1: diagrama PRISMA y *checklist*	El diagrama suma asignaciones y hay un *checklist* propio de 7 ítems	**WP9** diagrama PRISMA 2020 con razones y *checklist* PRISMA-ScR de 22 ítems	Incluir figura y *checklist* como material suplementario
R1-Mayor-3: cómo se comprobó la salida de la IA; duplicado de Perú	No hay validación humana; la deduplicación es exacta	**WP4** deduplicación difusa; **WP6** validación muestral	Párrafo de validación con métricas; eliminar el duplicado
R2-M1: cuántas decisiones fueron humanas	No se registra quién decide	**WP5** campo `decided_by` (llm_a, llm_b, human)	Informar del número de decisiones humanas
R2-M1: ¿solo *abstracts*?	MAP y la síntesis usan solo *abstracts* (truncados)	**WP8** campo `source_text_level` y texto completo OA opcional	Declararlo explícitamente
R2-M1: pasos incoherentes (5 vs 4)	El texto describe fases que en 2025 no existían	—	Alinear el texto con la Figura 1 actualizada
R2-M2: validar y retirar "objetivo"	No hay validación ni medición de estabilidad	**WP6** validación (sensibilidad, precisión, κ, estabilidad)	Retirar "objective" y "complete sweep"; añadir limitaciones

Comentario	Causa raíz en PALET	Cambio en la herramienta	Acción en el capítulo
R2-M2: la referencia clave no está publicada	Sin software citable	**WP13** *release* en Zenodo con DOI, `CITATION.cff` y, a medio plazo, un *software paper*	Citar el DOI de la versión usada
R2-M3: muchos papers fuera de tema; RQ4 y RQ5 escasas	Sin criterios de elegibilidad; búsqueda única; RQ3 sin ancla	**WP3** búsqueda por RQ; **WP5** cribado PCC	Re-cribar, informar de exclusiones y discutir la escasa cobertura
R2-M4: afirmaciones mal atribuidas	La síntesis no exige ni verifica citas	**WP8** verificador de afirmaciones	Fernando revisa cada cita apoyándose en la tabla de evidencia de **WP8**
R2-M5 / R1-Mayor-4: vincular los bloques con los papers	No hay extracción ni libro de códigos	**WP8** extracción y mapa de evidencia (bloque × paper)	Tabla de bloques con papers de apoyo y separación entre "revisión" y "lectura amplia"
R2-M6: "meta-analysis" → bibliometría	Etiqueta del informe	**WP9** sección "Descriptive bibliometrics"	Renombrar la sección 3
R2-menores: Fig. 2 y Fig. 3	Diagramas de caja con $n < 5$; leyenda "PI"; año parcial; sin fuente ni fecha de las citas	**WP9** gráficos de puntos si $n < 10$, etiquetas RQ, aviso de año parcial y pie con fuente y fecha	Rehacer las figuras
R2-menores: referencias citadas y no listadas, y al revés	Sin verificador	**WP10** `check_citations.py`	Ejecutar y corregir
R1-menor: Chain-of-Thought citado y en nota	Redacción	—	Dejar solo la cita (y comprobar el año de Wei et al.)
R1-menor: referencias de agentes antiguas	Redacción	—	Añadir referencias recientes de agentes LLM

5 Fortalezas (conservar)

- **Fases con nombre y artefactos por fase** (ARQ, QSP, SE, MAP y PRISMA/síntesis/informe), coherentes con la narrativa del capítulo. Es una buena base para la trazabilidad.
- ***Prompts* de calidad**: estructura XML, ejemplos *few-shot*, casos límite y salida JSON. El de síntesis tiene buenos principios: usar solo los papers aportados, dar recuentos de evidencia y separar confianza cuantitativa y cualitativa.
- **MAP con `temperature=0`, validación de esquema y reintentos**, y separación entre `unmatched_no_abstract` y `unmatched_not_relevant`.
- **SE**: reconstrucción del *abstract* desde el índice invertido de OpenAlex, normalización de DOI y esquema común OpenAlex/PubMed.
- ***Timestamp* unificado** propagado entre fases.

- **GUI** con progreso en tiempo real, historial, estimación de coste y una compuerta de aprobación tras ARQ. Es la semilla del humano en el bucle.
- **Plantilla de *seed prompt*** de 7 elementos (Tabla 1 del capítulo): ya es casi un protocolo. Solo falta convertirla en un objeto formal y congelado.
- **Publicación abierta** (MIT), `.env` fuera del control de versiones, `.env.example` y README con financiación UE.

6 Debilidades por módulo

Severidad: [C] invalida la revisión; [A] compromete la calidad o la defensa ante *referees*; [M] deuda técnica relevante; [B] menor.

6.1 ARQ (`scripts/arq_module.py`, `prompts/arq_system_v3.txt`)

- [C] **El LLM reescribe las RQs del investigador.** El *prompt* pide "generar" entre 1 y 5 RQs y `validate_arq_output` exige ese mismo rango. El capítulo tiene 7. En la ejecución `20260521_092213` el modelo **fusionó las RQ5 y RQ6 originales en una sola**. Las RQs deben ser entrada del usuario, inmutables. El LLM solo puede *sugerir* cambios, que el humano acepta de forma explícita.
- [A] **Criterios de inclusión y exclusión inventados y nunca aplicados.** El *prompt* pide "inferir" entre 2 y 4 criterios. Solo se muestran en la GUI y en el informe (`app.py:180`, `report_generator_v2.py:839`); ninguna fase los aplica.
- [A] **`search_filters` se ignoran.** Los filtros de año y tipo vienen de `argparse` o de la GUI, y el idioma no se aplica nunca.
- [M] Sin `temperature`, de modo que la salida varía entre ejecuciones. Contradicción de idioma: el caso límite dice "RQs en el idioma del *seed*" y el ejemplo 4 las da en inglés.
- [M] Las palabras clave son una lista plana (5–15), sin estructura por concepto ni por RQ, lo que provoca el problema de QSP.
- [B] `validate_arq_output` no se llama en el *pipeline*; hay `sys.exit` al importar el módulo; los precios están en el código.

6.2 QSP (`scripts/qsp_module.py`, `prompts/qsp_system_v2.txt`)

- [C] **Una sola cadena booleana para todas las RQs.** Con bloques unidos por AND, un término específico de una RQ restringe la recuperación de las demás, y las RQs minoritarias (educación, inmigración) quedan infrarrepresentadas por diseño.
- [A] **Esquemas contradictorios**: el mensaje de usuario pide `alternative_queries` como lista; el *system prompt*, como diccionario. `validation_status` y `warnings` nunca se comprueban, y no hay validador sintáctico.
- [A] Solo se ejecuta OpenAlex. La traducción a Scopus o WoS existe en el *prompt* pero no en el SE.
- [M] No se guardan ni el razonamiento ni las alternativas. `json.loads` falla si el texto no empieza exactamente por `{`.

6.3 SE (`scripts/se_module.py`)

- [C] **OpenAlex `search` busca en título, resumen, texto completo y palabras clave.** Las coincidencias en el texto completo traen trabajos que mencionan un término de pasada. La documentación vigente indica restringir con `search.title_and_abstract` (filtro `title_and_abstract.search`).
- [C] **Truncado silencioso.** Se cortan los resultados a `max_results` (100 por defecto; las ejecuciones reales usaron entre 10 y 40) ordenados por relevancia, y no se guarda `meta.count`. Lo que PRISMA llama "identificados" son en realidad "descargados". Con 20 resultados no hay "barrido completo" posible.
- [C] **Autenticación obsoleta.** Desde febrero de 2026 OpenAlex exige API key: el parámetro `mailto` se ignora y, sin clave, el presupuesto diario es mínimo. El código solo envía `mailto`. Hay que añadir `OPENALEX_API_KEY` y comprobar el límite vigente de `per_page` (la documentación recomienda 100; el código usa 200).
- [A] **Sin Scopus ni WoS**, aunque los *referees* preguntan por ellas y el capítulo usó Scopus. La API de Scopus es viable desde la IP del campus o la VPN con una API key gratuita de `dev.elsevier.com`, o desde fuera con un *InstToken* solicitado por la institución.
- [A] **Deduplicación insuficiente**: DOI exacto, y título exacto solo entre registros sin DOI. No detecta el caso Coaquira/Velásquez (§3).
- [A] No se registran la fecha y hora de la búsqueda, el idioma, el tipo de documento, los países de afiliación, `relevance_score` ni `primary_topic`. Usa `concepts`, un campo heredado de OpenAlex, en lugar de `topics`.
- [M] `sys.exit` dentro de la librería; sin reintentos ni *backoff*; **correo personal escrito en el código** como valor por defecto de `CONTACT_EMAIL` (repositorio público). PubMed apenas aporta en economía: dejarlo como opcional.

6.4 MAP (`scripts/map_module_v2.py`, `prompts/map_system_v2.txt`)

- [C] **MAP no criba.** Solo pregunta si el artículo responde a alguna RQ. No evalúa el contexto (UE), el concepto (empleo y transición) ni el tipo de documento. De ahí vienen los papers fuera de tema.
- [C] **Errores contados como exclusiones.** Si fallan la validación, el JSON o la API, se devuelve `assignments: []` y `generate_assignment_summary` lo cuenta como "no relevante" en PRISMA.
- [A] **La confianza baja se incluye igual**, y el ejemplo 005 del *prompt enseña* a incluir un *think piece* vago con confianza baja. Las "consistency guidelines" ("watch for drift") no tienen efecto en llamadas independientes.
- [A] Una sola pasada y un solo modelo, sin categoría *unclear*, sin humano y sin medir el acuerdo.
- [A] `paper_id` es el índice posicional del DataFrame, y luego se usa con `iloc` en síntesis e informe. Es frágil ante reordenaciones y filtrados.
- [M] Se parsea texto libre en lugar de usar salidas estructuradas (*tool use* o esquema JSON). No hay caché de respuestas. `batch_size` no agrupa nada: solo introduce una pausa.

6.5 PRISMA (`prisma_validator.py`)

- [C] `included` **suma asignaciones por RQ**, de modo que hay doble conteo si un paper tiene varias RQs. La corrección con un conjunto deduplicado de IDs **no está en esta copia**:

la línea `included = sum(len(v) ...)` sigue ahí.

- [A] El "*checklist*" tiene 7 ítems propios, no los 22 de PRISMA-ScR. El ítem "razones de exclusión" es siempre `True` y "duplicados reportados" (`>= 0`) también lo es siempre.
- [M] El diagrama está fijado en español, no sigue la plantilla PRISMA 2020 y le faltan cajas (registros no recuperados, excluidos con razones, fuentes adicionales).

6.6 Síntesis (`scripts/synthesis_module.py`, `prompts/synthesis_system.txt`)

- [C] **Sin citas por afirmación**: el JSON de salida es prosa sin IDs, así que es imposible auditarla. Es el mismo mecanismo que produjo M4/M5.
- [A] **Sesgo de selección**: solo los 25 papers más citados por RQ (las citas de OpenAlex favorecen lo antiguo), con resúmenes truncados a 1.200 caracteres. Solo se usan resúmenes.
- [A] El análisis de lagunas se basa en `concepts` de 10 papers por RQ y no distingue entre una laguna de la literatura y un artefacto de la búsqueda, como la escasez de RQ4 y RQ5.
- [M] No hay libro de códigos que vincule recomendaciones con bloques de política.

6.7 Informe y BibTeX

- [A] **`escape_latex` sigue con el error de orden**: la barra invertida se reemplaza la última y rompe las secuencias ya escapadas (`report_generator_v2.py:46`). La corrección no llegó a esta copia.
- [A] Clave fantasma `'unmatched'` (`report_generator_v2.py:885, 1082`; `palet.py`, resumen final): siempre vale 0.
- [M] Uso de `iloc` con `paper_id`; diagramas de caja con $n < 5$; recuentos de citas sin fuente ni fecha; año parcial sin aviso.
- [A] Claves BibTeX `apellido_año_idx`, inestables entre ejecuciones. Todo se exporta como `@article`, sin `journal` ni páginas en PubMed, y nada se contrasta con el manuscrito, que es de donde vienen los errores de referencias del capítulo.

6.8 GUI, orquestación e infraestructura

- [A] **Orquestación duplicada**: `palet.py` y `app.py` reimplementan el *pipeline* por separado, con el consiguiente riesgo de que diverjan.
- [A] **`/api/download/{filename:path}` permite leer cualquier fichero del disco** (*path traversal*). Solo escucha en local, pero hay que cerrarlo.
- [M] Estado global en memoria. La aprobación solo existe tras ARQ y no permite editar las RQs, los criterios ni la cadena de búsqueda.
- [M] **Sin tests ni CI**; los `test_*.py` son *scripts* de humo. Ficheros heredados mezclados con los activos (`palet_legacy.py`, `map_module.py`, `report_generator.py`, `se_module_legacy.py`). Tres entornos virtuales en la carpeta (`.venv`, `venv`, `palet_env`).
- [M] IDs de modelo escritos a mano en 5 sitios, y uno sigue en `claude-sonnet-4-20250514` (`map_module.py`). Los precios también están en el código.
- [M] **La afirmación "temperature=0 = reproducibilidad total" es falsa**: `temperature=0` no garantiza el determinismo y los modelos cambian. La reproducibilidad real exige caché de respuestas y un manifiesto.

- [B] El README tiene marcadores `YOUR_USERNAME`; el `user.email` de git sigue con un valor de relleno.

7 Arquitectura objetivo: PALET 2.0

7.1 Principios de diseño

P1. Protocolo primero, congelado y versionado.
Un `protocol.yaml` con las RQs del investigador, los criterios PCC (*Population, Concept, Context*, según la guía JBI de revisiones de alcance), las fuentes, el periodo, los idiomas y los tipos de documento. El LLM *sugiere* y el humano *aprueba*. Todo cambio posterior genera una nueva versión, con *diff*.

P2. Todo registro tiene estado y razón.
Máquina de estados: `identified` → `deduplicated` → `ta_screened` (include / exclude[*reason*] / unclear) → `adjudicated` → [`ft_screened`] → `included` → `mapped` → `extracted`. Nada desaparece en silencio, y un error es un estado (`error`), nunca una exclusión.

P3. Humano en el bucle donde importa.
En cuatro puntos: aprobación del protocolo, aprobación de la búsqueda, adjudicación de discrepancias y *unclear*, y validación muestral.

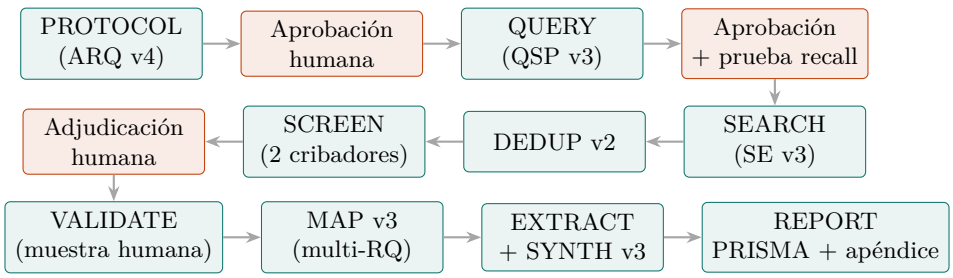
P4. Fiabilidad medida y declarada.
Recall de la búsqueda frente a un conjunto de referencias conocidas; sensibilidad, especificidad, precisión y κ del cribado frente a humanos; estabilidad entre ejecuciones y entre modelos.

P5. Síntesis anclada.
Cada afirmación lleva sus IDs y una cita literal *verificada por código*. Las afirmaciones que vienen de lectura externa se marcan como tales.

P6. Reproducibilidad por construcción.
Manifiesto de ejecución y caché *content-addressed* de llamadas a LLM y a API. Una re-ejecución con caché da resultados idénticos.

P7. Un solo motor.
Un paquete `palet/` que usan tanto la CLI como la GUI.

7.2 Flujo



Los nodos naranjas son puntos de control humano. El manifiesto de ejecución y la caché atraviesan todas las fases.

7.3 Estructura de directorios propuesta

```
palet/
  palet/                      # paquete (motor unico)
    __init__.py
    config.py                 # carga palet.toml (modelos, precios, limites)
    models.py                 # esquemas Pydantic (Apendice B)
    manifest.py               # manifiesto de ejecucion + hashes
    cache.py                  # cache content-addressed (LLM y API)
    llm.py                    # cliente Anthropic: structured outputs, retries, batches
    protocol.py               # ARQ v4
    query/                    # QSP v3: builder determinista + validadores
      builder.py  validate_scopus.py  validate_openalex.py
    sources/                  # adaptadores enchufables
      base.py  openalex.py  scopus.py  pubmed.py  citation_chasing.py
    dedup.py
    screen.py                 # SCREEN: criterio a criterio, 2 cribadores
    adjudicate.py             # export/import Excel para humanos
    validate.py               # metricas, IC, estabilidad, recall de busqueda
    mapping.py                # MAP v3
    extract.py  synthesize.py  verify_claims.py
    report/  prisma.py  checklist_prisma_scr.py  figures.py  latex.py  methods_appendix.py
    bib.py  check_citations.py
    pipeline.py               # orquestador unico (CLI y GUI lo usan)
  cli.py                      # sustituye a palet.py (mantener alias)
  app.py                      # FastAPI -> llama a palet.pipeline
  prompts/                    # versionados: screen_v1.txt, extract_v1.txt, ...
  tests/                      # pytest + fixtures grabadas (sin red)
  runs/<run_id>/              # TODO artefacto de una ejecucion vive aqui
    manifest.json protocol.yaml search_log.json records.parquet
    decisions.jsonl llm_cache/ report/
  legacy/                     # ficheros antiguos (no borrar: mover)
  palet.toml  pyproject.toml  CITATION.cff
```

8 Paquetes de trabajo (especificación para el agente)

Orden recomendado: **WP0** → **WP1** → **WP2** → **WP4** → **WP5** → **WP6** → **WP9** → **WP10** (**ruta mínima para la resubmisión del capítulo**) y después **WP3**, **WP7**, **WP8**, **WP11**, **WP12** y **WP13**. Cada WP termina con tests en verde y una nota en `CHANGELOG.md`.

WP0 — Higiene y red de seguridad (1–2 días)

Objetivo: congelar el estado actual, corregir los errores conocidos y montar los tests antes de refactorizar.

1. Crear la rama `refactor/palet-2`. Mover a `legacy/` (**sin borrar**) `palet_legacy.py`, `scripts/map_module.py`, `scripts/report_generator.py`, `scripts/se_module_legacy.py` y `prompts/*_system.txt` sin versión.
2. Crear `pyproject.toml` y `palet.toml`: IDs de modelo, precios por MTok, límites y `per_page`. Ningún ID de modelo ni precio puede quedar en el código.
3. Corregir `escape_latex`: primero la barra invertida, con un mapa de un solo paso (`re.sub` con diccionario), y añadir un test con `\ & % _ { } ~ ^ $ #` y caracteres cirílicos.
4. Corregir `included` en PRISMA con un conjunto deduplicado de IDs, y añadir un test con un paper asignado a dos RQs.
5. Eliminar la clave fantasma `'unmatched'` (`palet.py` y `report_generator_v2.py`) y usar `unmatched_not_relevant` y `unmatched_no_abstract`.

6. **Errores ≠ exclusiones**: en MAP, cualquier fallo devuelve `status="error"` y la pareja (paper, RQ) se reintenta o se marca para revisión. No cuenta como "no relevante".
7. Quitar el correo personal escrito en `se_module.py` (leerlo de `.env` sin valor por defecto) y cerrar el *path traversal* de `/api/download`: resolver la ruta y exigir que esté dentro de `results/` o `runs/`.
8. Quitar los `sys.exit` de los módulos de librería (lanzar excepciones) y los `input()` fuera de la CLI.
9. `tests/` con `pytest`: deduplicación, `escape_latex`, recuentos PRISMA, parseo de OpenAlex (*fixture* JSON grabada) y validación de esquemas. CI en GitHub Actions con `ruff` y `pytest`.

Aceptación: `pytest` en verde; el *pipeline* actual se ejecuta de principio a fin con `--max-results 10` y produce las mismas salidas que antes salvo las correcciones.

WP1 — Modelo de datos, manifiesto y caché (2–3 días)

1. Esquemas Pydantic (Apéndice B) para `Protocol`, `SearchQuery`, `SearchExecution`, `Record`, `ScreeningDecision`, `Adjudication`, `RQAssignment`, `ExtractionItem`, `Claim` y `RunManifest`.
2. **ID estable de registro**: `record_id` = DOI normalizado si existe; si no, `sha1(titulo_normalizado|primer` Prohibido usar el índice posicional. Eliminar todos los `iloc[paper_id]`.
3. `runs/<run_id>/` como única ubicación de los artefactos de una ejecución (hoy están repartidos entre `outputs/json`, `data/raw` y `results`).
4. `RunManifest`: *hash* de git, versión de PALET, `model_id` exacto por fase, parámetros, `sha256` de cada *prompt*, fecha y hora UTC de cada búsqueda, `meta.count` por consulta y tokens y coste reales.
5. `cache.py`: clave = `sha256(model|system|messages|params)`. Guarda la respuesta cruda. Con `--replay` no se llama a ninguna API y los resultados son idénticos.

Aceptación: dos ejecuciones con `--replay` producen `decisions.jsonl` idénticos byte a byte; el manifiesto contiene todos los campos.

WP2 — Protocolo (ARQ v4) y puntos de control (2 días)

1. Si el *seed* contiene RQs, **se copian literalmente** y quedan congeladas. El LLM devuelve `suggested_edits` separadas, nunca sustituciones. Sin límite de 5 RQs.
2. Criterios PCC estructurados: para cada criterio, `id`, `type` (inclusión o exclusión), `dimension` (population/concept/context/study_type/language/period), `rule` en lenguaje operativo y `examples_in`/`examples_out`.
3. `search_filters` se convierten en filtros de API reales (año, idioma, tipo) y quedan registrados.
4. Bloques conceptuales: el LLM propone sinónimos por concepto (no una lista plana), por ejemplo `transition`, `labour`, `geography` y un bloque específico de cada RQ.
5. El humano aprueba editando `protocol.yaml` (CLI) o un formulario (GUI). Queda registrado `approved_by` y `approved_at`.
6. `temperature=0` y `model_id` desde la configuración.

Aceptación: con el *seed* ISABEL de 7 RQs se obtienen exactamente 7 RQs idénticas al texto original; el `protocol.yaml` pasa la validación del esquema.

WP3 — Búsqueda (QSP v3 + SE v3) (4–6 días)

1. **Constructor determinista**: la cadena se construye *por código* a partir de los bloques conceptuales del protocolo. El LLM solo propone sinónimos. Estrategia: Q_k = TRANSICIÓN∧ LABORAL∧GEO∧BLOQUE$_k$ para cada RQ k, más una consulta general sin bloque específico. Se ejecutan todas, se unen los resultados y se guarda qué consulta recuperó cada registro.
2. **Traductores y validadores por base de datos**: OpenAlex (`search.title_and_abstract`, booleanos en mayúsculas, comillas en las frases) y Scopus (`TITLE-ABS-KEY(...)`, `PUBYEAR`, `LANGUAGE`, `DOCTYPE`, `AND NOT`, paréntesis equilibrados, ≤ límite de longitud). Tests unitarios con cadenas válidas e inválidas.
3. **OpenAlex**: `OPENALEX_API_KEY` obligatoria, cabeceras de presupuesto (`X-RateLimit-Remaining-USD`) registradas, `per_page` desde la configuración, paginación con `cursor=*`, reintentos con *backoff* ante 429 y 5xx. Guardar `meta.count`, `relevance_score`, `language`, `type`, `primary_topic`, `topics`, países de las instituciones y `referenced_works`.
4. **Sin truncado silencioso**: si `meta.count` supera el umbral configurado, se *para* y se pide refinar o confirmar la descarga completa. Nunca se corta en silencio. Si el humano decide limitar, queda registrado y se informa.
5. **Adaptador de Scopus** (`sources/scopus.py`): `X-ELS-APIKey` y, opcionalmente, `X-ELS-Insttoken`. Funciona desde la red de la Universidad de Oviedo o su VPN. Vista `COMPLETE` si está disponible (resúmenes); si no, recuperar los resúmenes por DOI.
6. **Importación manual** como alternativa: aceptar CSV o RIS exportados desde la web de Scopus o WoS (el caso del capítulo). La cadena y la fecha se introducen a mano y quedan en el registro de búsqueda.
7. **Prueba de recall de la búsqueda**: `protocol.yaml` admite `known_items` (20–40 DOIs que los autores consideran imprescindibles). Se informa del recall = encontrados / conocidos, y si es inferior a 0,8 se avisa y se bloquea la aprobación.
8. (Opcional) **Búsqueda de citas** hacia atrás y hacia delante de los incluidos vía `referenced_works` y `cites:` de OpenAlex, registrada como "otras fuentes" en PRISMA.
9. `search_log.json` con los campos de PRISMA-S: base de datos, plataforma, cadena exacta, filtros, fecha y hora, recuento total y recuento descargado.

Aceptación: con el protocolo ISABEL, cada RQ tiene su consulta, los recuentos totales quedan registrados y el recall de `known_items` se informa.

WP4 — Deduplicación v2 (1 día)

1. Pasada 1: DOI normalizado (minúsculas, sin prefijo `https://doi.org/`, sin espacios).
2. Pasada 2: título normalizado (minúsculas, sin acentos ni puntuación, y **partido por ";" o "["** para quedarse con la parte inglesa de los títulos bilingües) con similitud ≥ 0,92 (`rapidfuzz.token_set_ratio`) **y** solapamiento de apellidos de autores ≥ 1 **y** |Δaño| ≤ 1.
3. Los pares dudosos (similitud entre 0,85 y 0,92) se marcan para revisión humana; no se fusionan solos.
4. Al fusionar se conserva el registro con DOI y más metadatos, y se guarda `merged_from`.

Aceptación: test con el par Coaquira 2022 / Velásquez 2023, que debe fusionarse, y con dos falsos positivos plausibles, que no.

WP5 — SCREEN: cribado por criterios (4–5 días)

1. **Una llamada por registro** con salida estructurada (*tool use* o esquema JSON; Apéndice C). Para cada criterio del protocolo: `met` (yes/no/unclear), `evidence_quote` (cita literal del título o resumen, o `null`) y una justificación breve.
2. **Regla de decisión en código, no en el LLM**: incluir si se cumplen todos los criterios de inclusión y ninguno de exclusión; excluir si algún criterio es `no`, con `reason_code` igual al primero que falla según un orden fijo (Apéndice D); *unclear* en cualquier otro caso.
3. **Verificación de citas**: si `evidence_quote` no aparece en el texto (coincidencia difusa $\geq 0{,}9$), la decisión pasa a *unclear*.
4. **Dos cribadores independientes**: dos modelos distintos o el mismo con *prompts* distintos, configurable. Si coinciden, la decisión es firme. Si discrepan o alguno da *unclear*, pasa a adjudicación humana.
5. **Fase de título y resumen orientada a la sensibilidad**: ante la duda, *unclear*, nunca exclusión.
6. **Adjudicación humana** (`adjudicate.py`): exporta a Excel los registros pendientes, con título, resumen, decisiones de A y B, citas y una columna `human_decision` con lista desplegable. Al reimportar se registran `decided_by="human:<iniciales>"` y la fecha.
7. Registros sin resumen: intentar recuperar el resumen (Scopus por DOI u OpenAlex); si no se consigue, `reason_code=NO_ABSTRACT` y se informa por separado (no son "no relevantes").

Aceptación: el 100% de los registros tiene estado final y razón; existen las salidas `decisions.jsonl` y `screening_summary.csv`; re-cribar el corpus del capítulo (234 registros) genera razones para todas las exclusiones.

WP6 — VALIDATE: fiabilidad medida (2–3 días de código y 1–2 días humanos)

1. **Muestra**: todos los incluidos por PALET más una muestra aleatoria simple de los excluidos (o estratificada por razón), hasta unos 150–200 registros, con semilla fija. Se exporta a Excel **sin las decisiones de PALET** (a ciegas).
2. **Dos humanos** (p. ej., Ricardo y Fernando, o un colega) criban de forma independiente con los mismos criterios. Las discrepancias se resuelven por consenso y forman el patrón de referencia.
3. **Métricas**: sensibilidad (recall), especificidad, precisión (VPP) y F1 de PALET frente al patrón; κ de Cohen entre humanos y entre PALET y el patrón; IC del 95% (Wilson para proporciones, *bootstrap* para κ). Si la muestra de excluidos es aleatoria, la sensibilidad se extrapola ponderando por la fracción muestreada.
4. **Estabilidad**: re-cribar sin caché 3 veces (o con 2 modelos) un subconjunto de 100 registros e informar del porcentaje de acuerdo y de la κ de Fleiss.
5. Salida: `validation_report.json` y una tabla LaTeX lista para el capítulo.

Aceptación: la tabla de validación se genera automáticamente e incluye los IC. Si la sensibilidad es inferior a 0,90, el informe lo marca en rojo y sugiere ajustar los criterios o el *prompt*.

WP7 — MAP v3 (1–2 días)

- Solo se aplica a los registros *incluidos*. Asignación múltiple explícita (o única, si el protocolo lo fija).

- Salida por pareja (registro, RQ): `relevance` (core/partial/none) más `evidence_quote`. La relevancia `partial` se informa por separado.
- Corregir el *prompt*: eliminar el ejemplo 005 (inclusión con confianza baja) y la sección de "drift". Cada RQ incorpora su contexto, por ejemplo "in the EU".

WP8 — Extracción y síntesis anclada (4–6 días)

1. **EXTRACT** por registro incluido: lista de `ExtractionItem` con `recommendation`, `policy_domain` (libro de códigos configurable; para ISABEL, los seis bloques del capítulo), `evidence_type` (empírica, teórica, revisión o política), `geography`, `quote` literal y `source_text_level` (abstract o full_text).
2. **Texto completo opcional**: si hay versión OA (`best_oa_location` de OpenAlex o Unpaywall), extraer de ella y marcarlo así.
3. **SYNTH v3**: recibe *todos* los `ExtractionItem` de un bloque o RQ, no los 25 más citados. Devuelve frases como objetos `Claim`: `text`, `supporting_record_ids` (mínimo 1), `support_quotes` y `origin` (review o external).
4. `verify_claims.py`: comprueba que cada cita aparece en el texto del registro (difusa $\geq 0{,}9$) y que la afirmación está respaldada por esa cita, mediante una segunda llamada LLM "juez" con salida supported/partial/unsupported. Las afirmaciones no respaldadas se eliminan o se marcan en rojo.
5. **Mapa de evidencia**: tabla bloque × registros, con el número de estudios por bloque y por tipo de evidencia. Es justo la tabla que piden R1 y R2-M5.
6. **Análisis de lagunas**: separar las lagunas de la literatura de los artefactos de búsqueda, por ejemplo cuando la consulta del bloque de una RQ tuvo un recuento total bajo.

Aceptación: 0 afirmaciones sin IDs; al menos el 95% de las citas verificadas; el mapa de evidencia se exporta en LaTeX y CSV.

WP9 — Informe, PRISMA-ScR y apéndice de métodos (3 días)

1. **Diagrama PRISMA 2020** (en inglés por defecto, idioma configurable): registros identificados por base de datos y por otras fuentes, duplicados eliminados, cribados, excluidos por razón (con desglose), no recuperables, evaluados a texto completo e incluidos. Los recuentos se calculan de `records` (estado final), no de `summary`. Exportar en PDF vectorial y en PNG a 600 ppp.
2. ***Checklist* PRISMA-ScR de 22 ítems**: cada ítem con estado `auto` (PALET lo rellena: fuentes, búsqueda, selección, flujo, etc.) o `manual` (los autores deben completarlo: protocolo y registro, evaluación crítica, etc.), con la página o sección. Nada se da por cumplido sin evidencia.
3. **Apéndice de métodos automático** (inglés, `methods_appendix.tex`): modelos y versiones, fechas, cadenas exactas, filtros, criterios, número de decisiones humanas y del LLM, métricas de validación y *prompts* (*hash* y texto en material suplementario).
4. **Declaración de uso de IA** alineada con RAISE: qué fases usaron IA, con qué supervisión humana, cómo se validó y cuáles son sus limitaciones.
5. **Figuras**: etiquetas RQ (nunca "PI"); si $n < 10$ en algún grupo, gráfico de puntos con *jitter* en lugar de diagrama de caja; pie con "Citation counts: OpenAlex/Scopus, retrieved YYYY-MM-DD"; año parcial marcado; renombrar "meta-analysis" a "descriptive bibliometrics".

WP10 — Bibliografía y verificador de citas del manuscrito (1–2 días)

1. Claves BibTeX estables: `ApellidoAñoPalabraTítulo` (estilo Better BibTeX), con sufijo a/b determinista. Tipo de entrada según `type`.
2. `check_citations.py manuscrito.docx|.tex referencias.bib`: extrae las citas autor-año del texto, empareja con la lista de referencias o el `.bib` y produce cuatro listados: citadas y no listadas, listadas y no citadas, discrepancias de año o grafía ("Kapetaki et al. 2021" frente a "Kapetaki and Alves Dias 2021") y duplicados a/b.

Aceptación: aplicado al borrador del capítulo, reproduce al menos los casos señalados por el R2 (Green et al. 1997, Dijkstra et al. 2018, Galgóczi y Pochet 2019, la lista de no citadas, etc.).

WP11 — GUI y orquestación única (3–4 días)

- `app.py` y la CLI llaman a `palet.pipeline.run(...)`, que se implementa una sola vez.
- Puntos de control editables en la GUI: protocolo (RQs, criterios, bloques), búsqueda (cadenas y recuentos totales antes de descargar) y adjudicación (tabla con filtros, o Excel).
- Estado persistente por ejecución (`runs/<id>/state.json`) en lugar de una variable global; se puede reanudar una ejecución interrumpida.
- Panel de validación con métricas e IC, y panel PRISMA.

WP12 — Coste y rendimiento (1–2 días)

- **Message Batches API** para SCREEN, MAP y EXTRACT: más barata y sin `sleep`. *Prompt caching* del *system prompt* y del protocolo.
- Escalones de modelo configurables, por ejemplo: cribador A = `claude-sonnet-5-5`, cribador B = `claude-haiku-5-5`, juez y síntesis = `claude-opus-5-5`. Fijar siempre el ID exacto en el manifiesto y **no cambiar de modelo a mitad de una revisión**.
- Estimación de coste previa con los recuentos reales y un tope configurable (`max_cost_eur`).

WP13 — Documentación, citabilidad y banco de pruebas (2–3 días)

- `CITATION.cff` y *release* etiquetada con DOI de Zenodo, para que el capítulo cite una versión concreta (responde a "unpublished working paper").
- `docs/METHODS.md`: descripción metodológica con referencias, apta para pegar en artículos.
- **Banco de pruebas**: el corpus del capítulo, con el patrón humano de **WP6**, se guarda como *benchmark* versionado. Cada cambio de *prompt* o de modelo se evalúa contra él (sensibilidad, precisión y κ) antes de usarse en producción.
- Medio plazo: *software paper* (p. ej., JOSS) o artículo metodológico con la validación.

9 Plan para la revisión del capítulo

9.1 Dos opciones para el corpus

	A. Re-cribar los 234 registros existentes	**B. Nueva búsqueda documentada (recomendada si hay tiempo)**
Qué se hace	**WP0**+**WP4**+**WP5**+**WP6**+**WP9** sobre el CSV de Scopus importado	Lo mismo más **WP2**+**WP3**: búsqueda en Scopus (campus o VPN) y OpenAlex por RQ, con fecha
Responde a M1	Parcialmente: la cadena original solo si se recupera del historial de Scopus. Si no, declararlo con honestidad	Totalmente: cadenas, fechas y recuentos exactos
Responde a M3 (RQ4 y RQ5)	No: solo puede discutir la escasez como limitación	Sí: las consultas por RQ deberían recuperar educación e inmigración
Coste en tiempo	1–2 semanas	3–4 semanas
Riesgo	El R2 puede insistir en la búsqueda	La Sección 4 cambia y Fernando debe re-anclar los bloques

Una vía intermedia razonable: **A más búsquedas suplementarias documentadas solo para RQ4 y RQ5**, declaradas como tales en el diagrama PRISMA ("records identified through supplementary searches"). Es una decisión de los autores.

9.2 Pasos (opción A o intermedia)

1. **Recuperar la cadena original** de Scopus (historial de búsqueda de la cuenta, correo de exportación o notas de mayo de 2025). Si no aparece, decirlo.
2. Importar `Final_Fernando_3.1.csv` (235 registros), localizar el registro perdido y deduplicar con **WP4**. Resultado esperado: 1 duplicado (Perú).
3. Fijar los criterios PCC con Fernando (propuesta en el Apéndice E).
4. Re-cribar con **WP5**, adjudicar con **WP5**.6 y validar con **WP6** (Ricardo y Fernando a ciegas).
5. Re-asignar las RQs con **WP7**, aplicar extracción y mapa de evidencia con **WP8** y entregar el mapa a Fernando para revisar M4 y M5 cita por cita.
6. Generar el diagrama PRISMA, el *checklist*, el apéndice y las figuras con **WP9**. Ejecutar `check_citations.py` sobre el manuscrito.

9.3 Plantillas de texto para el capítulo (inglés; completar tras la re-ejecución)

Methods (reproducibility paragraph) — borrador

The search was run in Scopus (title, abstract and keywords) on [DATE] using the search string reported in Appendix A ([N_ID] records), restricted to [YEARS], [LANGUAGES] and [DOCUMENT TYPES]. PALET v[X.Y] (Zenodo DOI [..]) was used for de-duplication, title-and-abstract screening and question mapping, with Anthropic Claude models ([EXACT MODEL IDs]) called through the API at temperature 0. Each record was assessed against pre-specified eligibility criteria (Appendix B) by two independent model configurations; disagreements and uncertain cases ([N_H] records) were adjudicated by the authors. Screening and extraction were based on [titles and abstracts / open-access full texts]. The PRISMA-ScR flow diagram and checklist are provided in Figure [X] and Supplementary Material [Y].

Validation and limitations — borrador

To assess the reliability of the AI-assisted screening, two authors independently and blindly screened a random sample of [N] records (all [n_inc] included and [n_exc] excluded records). Against the consensus human decision, PALET achieved a sensitivity of [..] (95% CI [..]), a precision of [..] (95% CI [..]) and Cohen's κ of [..]. Repeated runs agreed in [..]% of decisions. The procedure is AI-assisted rather than objective: it may miss relevant studies (especially those not indexed in the searched database or described with different terminology), its outputs depend on the model version, and screening on abstracts may misclassify studies whose relevance is only apparent in the full text. Evidence for [RQ4 and RQ5] is thin ([n] and [n] studies), and conclusions for these blocks should be read as indicative.

Frases que hay que retirar del capítulo: "objective", "ensures a complete sweep of key references", "fully automated", "brief meta-analysis" y toda descripción de pasos que no se ejecutaron (filtrado PRISMA o síntesis contextualizada en la versión de 2025).

Respuesta a la parte de agentes (R1): reducir Wooldridge y Jennings, Shoham y Green et al. a un párrafo, como pide el R2, y añadir referencias recientes de agentes basados en LLM (p. ej., Yao et al., ReAct, ICLR 2023; Wang et al., "A survey on large language model based autonomous agents", *Frontiers of Computer Science*, 2024). Comprobar las referencias exactas antes de citarlas. En cuanto a Chain-of-Thought, quitar la nota al pie y dejar solo la cita, y revisar el año de Wei et al. (NeurIPS 2022).

10 Qué significa "una herramienta de 10"

Criterio verificable	Umbral
Recall de la búsqueda frente a `known_items`	$\geq 0{,}90$
Sensibilidad del cribado de título y resumen frente al patrón humano	$\geq 0{,}95$ (IC informado)
Precisión del cribado	$\geq 0{,}50$ (informada siempre)
κ PALET–patrón humano	$\geq 0{,}70$ (informada siempre)
Registros con estado final y razón	100%
Afirmaciones de síntesis con IDs y cita verificada	100% con IDs y $\geq 95\%$ verificadas
Re-ejecución con `--replay`	idéntica byte a byte
Checklist PRISMA-ScR	22/22 ítems con estado y ubicación
Cobertura de tests del paquete `palet/`	$\geq 70\%$; CI en verde
IDs de modelo, *prompts* y fechas en el manifiesto	100%
Versión citable (DOI)	sí

La literatura reciente sobre cribado con LLM muestra un rendimiento muy variable entre revisiones y advierte de que todavía no se recomienda automatizar por completo el cribado de título y resumen. La sensibilidad es la métrica crítica. Por eso el diseño propuesto no promete automatización total: **promete una asistencia medida, con un humano responsable**, que es lo que exigen la declaración conjunta Cochrane/Campbell/JBI/CEE y RAISE.

11 Instrucciones operativas para el agente implementador

Reglas de trabajo

1. **Idioma**: habla con Ricardo en español. El código, los identificadores, los *docstrings* y los mensajes de *commit* van en inglés. Los textos de interfaz pueden seguir en español.
2. **Ritmo**: Ricardo trabaja de forma secuencial y quirúrgica. Haz **un WP cada vez**; dentro de cada WP, un fichero o cambio coherente cada vez. Al final de cada paso, da un resumen breve (qué cambió, por qué y cómo probarlo) con el comando exacto que debe ejecutar, y espera su confirmación (suele responder "continuar").
3. **Entorno**: macOS en Apple Silicon, Python 3.12 de Homebrew (nativo arm64), VS Code y MacTeX. Usa un único entorno virtual `.venv` y no toques `venv/` ni `palet_env/`: propón eliminarlos y deja que Ricardo decida.
4. **No borres nada**: mueve a `legacy/`. No modifiques `results/`, `outputs/` ni `data/raw/` históricos; escribe todo lo nuevo en `runs/`.
5. **Secretos**: nunca escribas claves ni correos en el código. Variables nuevas en `.env.example`: `OPENALEX_API_KEY`, `ELSEVIER_API_KEY`, `ELSEVIER_INSTTOKEN` (opcional) y `CONTACT_EMAIL`.
6. **Coste**: antes de cualquier ejecución real con LLM de más de 1 € (estimado), muestra la estimación y pide permiso. En desarrollo usa `--max-records 20` y la caché.
7. **Tests primero** en las correcciones de errores: escribe el test que falla, corrige y comprueba que pasa.
8. **Compatibilidad**: mantén `python palet.py --seed-prompt ...` como alias de la nueva CLI hasta terminar **WP11**.
9. **Verifica antes de afirmar**: este informe cita líneas y ficheros de la versión `da90731`. Si algo no coincide, dilo y no supongas.
10. **Decisiones de método** (criterios PCC, opción A o B, umbrales, asignación única o múltiple): son de Ricardo y Fernando. Propón, no decidas.

A Lista de errores con ubicación

Sev.	Ubicación	Problema y solución
[C]	`prisma_validator.py`, `build_prisma_data`	`included` suma longitudes por RQ → usar `len(set().union(*rq_lists))`
[C]	`map_module_v2.py`, `analyze_paper` (ramas de error)	El error devuelve `assignments: []` y cuenta como "no relevante" → `status="error"`
[C]	`se_module.py`, `search_openalex`	`search` incluye el texto completo; sin `api_key`; truncado sin `meta.count`

Sev.	Ubicación	Problema y solución
[C]	`arq_module.py` y `arq_system_v3.txt`	Reescribe y limita las RQs a 1–5
[A]	`report_generator_v2.py`: 46	Orden de `escape_latex` (barra invertida al final)
[A]	`report_generator_v2.py`: 885, 1082; `palet.py` (resumen)	Clave fantasma `'unmatched'`
[A]	`report_generator_v2.py`: 141, 1026, 1094; `synthesis_module.py`	`iloc[int(paper_id)]` posicional → `record_id`
[A]	`se_module.py`, `remove_duplicates`	Solo DOI exacto o título exacto
[A]	`qsp_module.py`, `suggest_query_structure`	Esquema contradictorio con el *system prompt*; `json.loads` sin limpieza robusta; sin validación
[A]	`app.py`, `/api/download`	*Path traversal*
[A]	`app.py` y `palet.py`	Orquestación duplicada
[A]	`export_bibtex.py`	Claves inestables; todo como `@article`
[M]	`se_module.py:20`	Correo personal como valor por defecto en un repositorio público
[M]	Varios (5 ficheros)	`model=` escrito a mano; `map_module.py` con `claude-sonnet-4-20250514`
[M]	`arq_module.py`, `qsp_module.py`	Sin `temperature`; `sys.exit` al importar
[M]	`map_system_v2.txt`	Ejemplo 005 (incluye con confianza baja); sección “drift”
[M]	`synthesis_module.py`	Top 25 por citas; resúmenes truncados a 1.200 caracteres; sin citas por afirmación
[B]	`README.md`	`YOUR_USERNAME`; afirmación “reproducibility” con `temperature=0`

B Esquemas de datos (Pydantic v2)

```python
from datetime import datetime
from typing import Literal, Optional
from pydantic import BaseModel, Field

Dimension = Literal["population", "concept", "context",
                    "study_type", "language", "period"]

class Criterion(BaseModel):
    id: str                       # "IC1", "EC3"
    kind: Literal["include", "exclude"]
    dimension: Dimension
    rule: str                     # operational wording
    examples_in: list[str] = []
    examples_out: list[str] = []

class ConceptBlock(BaseModel):
    name: str                     # "transition", "labour", "geo", "rq4_education"
    terms: list[str]
    applies_to_rqs: list[str] = []   # [] = all

class Protocol(BaseModel):
    version: int
    title: str
    research_questions: dict[str, str]          # {"RQ1": "...", ...} frozen
```

```
    criteria: list[Criterion]
    concept_blocks: list[ConceptBlock]
    sources: list[Literal["scopus", "openalex", "pubmed", "wos", "manual_import"]]
    year_from: Optional[int] = None
    year_to: Optional[int] = None
    languages: list[str] = ["en"]
    document_types: list[str] = ["article", "review"]
    known_items: list[str] = []                # DOIs for search-recall test
    assignment_mode: Literal["single", "multi"] = "multi"
    approved_by: Optional[str] = None
    approved_at: Optional[datetime] = None

class SearchExecution(BaseModel):
    query_id: str               # "Q_RQ4_scopus"
    source: str
    platform: str               # "Scopus Search API", "OpenAlex API", "Scopus web export"
    query_string: str
    filters: dict
    executed_at: datetime
    total_hits: int             # meta.count / opensearch:totalResults
    downloaded: int
    truncated: bool
    truncation_approved_by: Optional[str] = None

class Record(BaseModel):
    record_id: str              # DOI or sha1(title|first_author|year)[:12]
    doi: Optional[str]
    title: str
    abstract: Optional[str]
    authors: list[str]
    year: Optional[int]
    source_title: Optional[str]
    language: Optional[str]
    doc_type: Optional[str]
    affiliation_countries: list[str] = []
    found_by_queries: list[str]
    merged_from: list[str] = []
    status: Literal["identified", "duplicate", "ta_included", "ta_excluded",
                    "ta_unclear", "ft_excluded", "included", "error"]

class CriterionJudgement(BaseModel):
    criterion_id: str
    met: Literal["yes", "no", "unclear"]
    evidence_quote: Optional[str]
    quote_verified: bool = False
    rationale: str = Field(max_length=300)

class ScreeningDecision(BaseModel):
    record_id: str
    screener: str               # "llm_a:claude-sonnet-5-5" | "human:RMVP"
    judgements: list[CriterionJudgement]
    decision: Literal["include", "exclude", "unclear", "error"]
    reason_code: Optional[str]  # see reason codes appendix
    prompt_sha256: str
    decided_at: datetime

class ExtractionItem(BaseModel):
    record_id: str
    recommendation: str
    policy_domain: str          # codebook value
    evidence_type: Literal["empirical", "theoretical", "review", "policy_document"]
    geography: list[str]
    quote: str
    quote_verified: bool
```

```
    source_text_level: Literal["abstract", "full_text"]

class Claim(BaseModel):
    text: str
    supporting_record_ids: list[str] = Field(min_length=1)
    support_quotes: list[str]
    origin: Literal["review", "external"]
    verdict: Optional[Literal["supported", "partial", "unsupported"]] = None
```

C Esqueleto del *prompt* de cribado

```
<role>You screen records for a scoping review. You do NOT decide inclusion;
you judge each eligibility criterion separately. Code applies the decision rule.</role>

<protocol>
Review title: {title}
Research questions (context only, do not screen on them): {rqs}
Criteria:
{for each criterion: id | include/exclude | dimension | rule | examples_in | examples_out}
</protocol>

<instructions>
1. Read ONLY the title and abstract given. Do not use outside knowledge about the paper.
2. For EVERY criterion return met = yes | no | unclear.
   - "yes"/"no" REQUIRE an evidence_quote copied verbatim from title/abstract.
   - If the text does not let you decide, answer "unclear" (never guess "no").
3. Geography: a study is in scope only if its empirical or policy focus includes
   the EU/EU member states (as defined in the protocol). Global studies count only
   if they report EU-specific results.
4. Keep rationale <= 40 words.
</instructions>

<output> Call the tool ‘record_screening‘ with the JSON schema provided. </output>
```

D Códigos de razón de exclusión (orden de precedencia)

Código	**Significado**	**Ejemplo en el corpus**
DUPLICATE	Duplicado fusionado	Velásquez 2023 / Coaquira 2022
WRONG_DOC_TYPE	Tipo de documento fuera del protocolo	editorial, nota
WRONG_LANGUAGE	Idioma fuera del protocolo	—
OUT_OF_PERIOD	Fuera del periodo	—
WRONG_CONTEXT	Fuera del ámbito geográfico (no UE)	Ye et al. 2014 (China)
WRONG_CONCEPT	Sin dimensión de empleo o mercado laboral	Jedelhauser y Binder 2015
WRONG_PHENOMENON	Sin vínculo con la transición verde o digital	Pobedin 2024
NO_POLICY_CONTENT	Sin recomendaciones ni análisis de política (si el protocolo lo exige)	—
NO_ABSTRACT	Sin resumen recuperable (se informa aparte)	—

E Propuesta de protocolo para el capítulo ISABEL (a validar por los autores)

```
version: 1
title: "Policies for job creation and destruction linked to the EU twin transition"
research_questions:            # copied verbatim from Table 2 of the chapter
  RQ1: "What economic-policy recommendations does the literature formulate to mitigate job
    destruction and boost job creation associated with the green transition in the EU?"
  RQ2: "What recommendations does the literature provide regarding the impact of digitalisation
    and artificial intelligence on JCD in the EU?"
  RQ3: "What cohesion or smart-specialisation policies does the evidence suggest?"   # consider
    adding: "to address JCD from the twin transition in EU regions"
  RQ4: "Which initial-education and lifelong-learning models are most effective for adapting green
     and digital skills?"
  RQ5: "What role do immigration and labour mobility play in filling skill gaps during the
    transition?"
  RQ6: "How should just-transition policies be designed to prevent green and digital discontent in
     territories that depend on fossil fuels or have low levels of digitalisation?"
  RQ7: "What knowledge gaps persist regarding the measurement and forecasting of JCD and the
    effectiveness of the cited policies?"
assignment_mode: multi
criteria:
  - {id: IC1, kind: include, dimension: context,
     rule: "Empirical or policy focus on the EU, EU member states or their regions (UK pre-2020
    allowed: DECIDE)"}
  - {id: IC2, kind: include, dimension: concept,
     rule: "Addresses employment, jobs, skills, labour markets or labour mobility"}
  - {id: IC3, kind: include, dimension: concept,
     rule: "Linked to the green and/or digital transition (decarbonisation, energy transition,
    automation, AI, digitalisation)"}
  - {id: IC4, kind: include, dimension: study_type,
     rule: "Peer-reviewed article or review"}
  - {id: EC1, kind: exclude, dimension: context,
     rule: "Exclusively non-European setting without EU-specific results"}
  - {id: EC2, kind: exclude, dimension: concept,
     rule: "Purely environmental/technical analysis without labour-market or policy dimension"}
year_from: 2013
year_to: 2025
languages: [en, es]
document_types: [article, review]
sources: [scopus, openalex]     # or: [manual_import] for option A
known_items: []                 # FILL: 20-40 DOIs the authors consider essential
```

F Referencias y fuentes externas

- Tricco, A. C. et al. (2018). PRISMA Extension for Scoping Reviews (PRISMA-ScR): checklist and explanation. *Annals of Internal Medicine*, 169(7), 467–473. doi:10.7326/M18-0850.
- Page, M. J. et al. (2021). The PRISMA 2020 statement. *BMJ*, 372, n71. doi:10.1136/bmj.n71.
- Rethlefsen, M. L. et al. (2021). PRISMA-S: an extension to the PRISMA statement for reporting literature searches in systematic reviews. *Systematic Reviews*, 10, 39. doi:10.1186/s13643-020-01542-z.
- Peters, M. D. J. et al. (2020). Updated methodological guidance for the conduct of scoping reviews. *JBI Evidence Synthesis*, 18(10), 2119–2126. doi:10.11124/JBIES-20-00167.
- Flemyng, E. et al. (2025). Position statement on artificial intelligence (AI) use in evidence synthesis across Cochrane, the Campbell Collaboration, JBI and the Collaboration for Environmental Evidence 2025. *Campbell Systematic Reviews*, 21, e70074. doi:10.1002/cl2.70074. Se apoya en las recomendaciones RAISE.
- Huotala, A., Kuutila, M. y Mäntylä, M. (2025). SESR-Eval: Dataset for evaluating LLMs in the title-abstract screening of systematic reviews. arXiv:2507.19027.

- Haddaway, N. R. et al. (2022). PRISMA2020: An R package and Shiny app for producing PRISMA 2020-compliant flow diagrams. *Campbell Systematic Reviews*, 18, e1230. doi:10.1002/cl2.1230.
- OpenAlex: documentación de búsqueda (`https://help.openalex.org/api/searching`) y de autenticación y precios (`https://developers.openalex.org/api-reference/authentication`). Consultadas el 11/11/2026.
- Elsevier Developer Portal: Scopus Search API y autenticación por IP o *InstToken* (`https://dev.elsevier.com`). Consultado el 11/11/2026.